SECANT METHOD

Find the root of equation $f(x) = x^2 - 4x - 10$ that lies between 1 and 4?

Matlab code

MAIN FUCTION

```
function main_secant

clc;

clear;

f = @(x) x.^2 - 4*x - 10;


x0 = 1;

x1 = 4;


tol = 1e-6;


maxIter = 100;


root = secant(f, x0, x1, tol, maxIter);


fprintf('Root = %.6f\n', root);


end


function root = secant(f, x0, x1, tol, maxIter)


for i = 1:maxIter
```

```
x2 = x1 - (f(x1)*(x1 - x0))/(f(x1)-f(x0));

fprintf('Iteration %d: x = %.6f\n', i, x2);

if abs(x2 - x1) < tol
root = x2;
return;
end

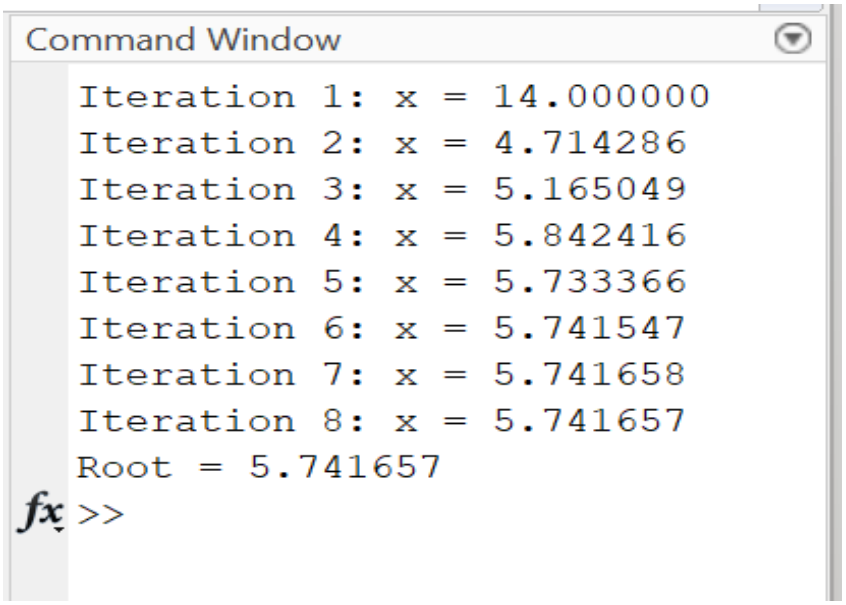
x0 = x1;
x1 = x2;

end

root = x2;

end
```

DISPLAY



COMBINED MAIN AND LOCAL FUNCTION

```
function main_secant

clc;

clear;


f = @(x) x^2 - 4*x - 10;


x0 = 1;

x1 = 4;


tol = 1e-6;

maxIter = 100;


root = secant(f, x0, x1, tol, maxIter);


fprintf('\nRoot = %.6f\n', root);


end


function root = secant(f, x0, x1, tol, maxIter)


for i = 1:maxIter


x2 = x1 - (f(x1)*(x1 - x0))/(f(x1) - f(x0));


fprintf('Iteration %d : x = %.6f\n', i, x2);
```

```
if abs(x2 - x1) < tol

root = x2;

return;

end


x0 = x1;

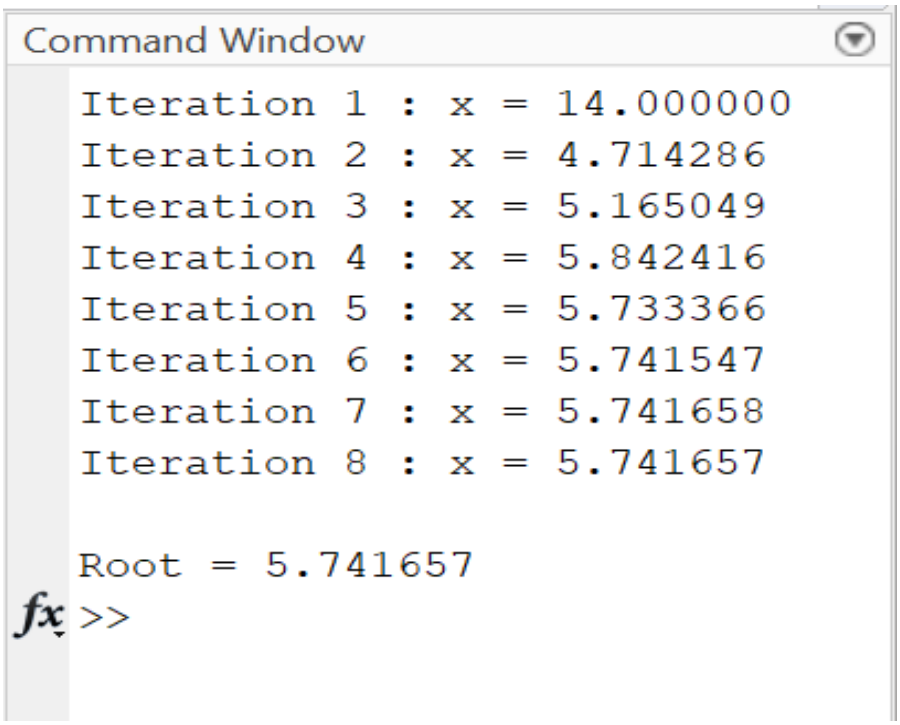
x1 = x2;


end


root = x2;


end
```

DISPLAY



COMBINED MAIN,LOCAL AND NESTED FUNCTION

```
function main_secant

clc;

clear;

x0 = 1;

x1 = 4;

tol = 1e-6;

maxIter = 100;


root1 = nestedSecant(x0,x1,tol,maxIter);


fprintf('\nRoot from Nested Function = %.6f\n',root1);


root2 = localSecant(x0,x1,tol,maxIter);


fprintf('Root from Local Function = %.6f\n',root2);


function root = nestedSecant(x0,x1,tol,maxIter)


for i = 1:maxIter


f0 = x0^2 - 4*x0 - 10;

f1 = x1^2 - 4*x1 - 10;


x2 = x1 - (f1*(x1-x0))/(f1-f0);
```

```
if abs(x2-x1) < tol
root = x2;
return;
end

x0 = x1;
x1 = x2;

end

root = x2;

end

end

function root = localSecant(x0,x1,tol,maxIter)

for i = 1:maxIter

f0 = x0^2 - 4*x0 - 10;
f1 = x1^2 - 4*x1 - 10;

x2 = x1 - (f1*(x1-x0))/(f1-f0);

if abs(x2-x1) < tol
```

```
root = x2;

return;

end


x0 = x1;

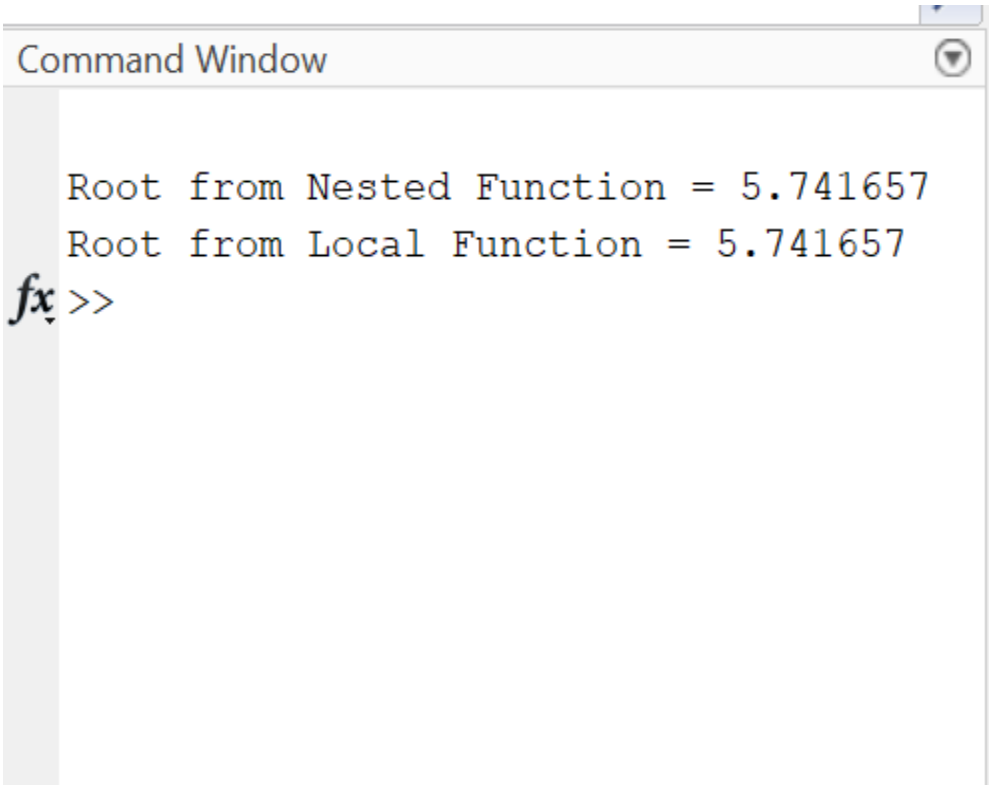
x1 = x2;


end


root = x2;


end
```

DISPLAY



NEWTON RAHPSON METHOD

Use Newton-Raphson method to solve the equation x3-7x2+5x-4=0 near x0 =5 correct to 8 decimal places.

Matlab code

MAIN FUNCTION

```
function main_newtonraphson

clc;

clear;


x0 = 5;


tol = 1e-8;

maxIter = 100;


for i = 1:maxIter


f = x0^3 - 7*x0^2 + 5*x0 - 4;

df = 3*x0^2 - 14*x0 + 5;


x1 = x0 - f/df;


fprintf('Iteration %d : x = %.8f\n', i, x1);


if abs(x1 - x0) < tol

break;

end
```

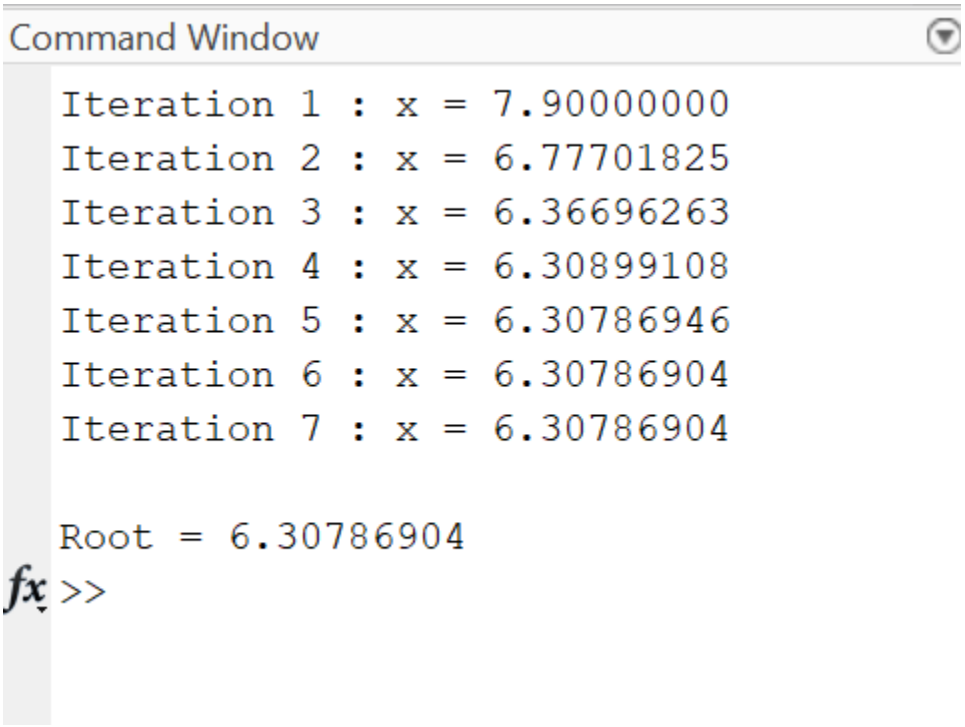
```
x0 = x1;


end


fprintf('\nRoot = %.8f\n', x1);


end
```

DISPLAY



COMBINED MAIN, LOCAL AND NESTED FUNCTION

```
function main_newton

clc;

clear;


x0 = 5;


tol = 1e-8;

maxIter = 100;


root1 = nestedNewton(x0,tol,maxIter);

fprintf('\nRoot from Nested Function = %.8f\n',root1);


root2 = localNewton(x0,tol,maxIter);

fprintf('Root from Local Function = %.8f\n',root2);



function root = nestedNewton(x0,tol,maxIter)


for i = 1:maxIter


f = x0^3 - 7*x0^2 + 5*x0 - 4;

df = 3*x0^2 - 14*x0 + 5;


x1 = x0 - f/df;
```

```
if abs(x1-x0) < tol
root = x1;
return;
end

x0 = x1;

end

root = x1;

end

end


function root = localNewton(x0,tol,maxIter)

for i = 1:maxIter

f = x0^3 - 7*x0^2 + 5*x0 - 4;
df = 3*x0^2 - 14*x0 + 5;

x1 = x0 - f/df;

if abs(x1-x0) < tol
```

```
root = x1;

return;

end

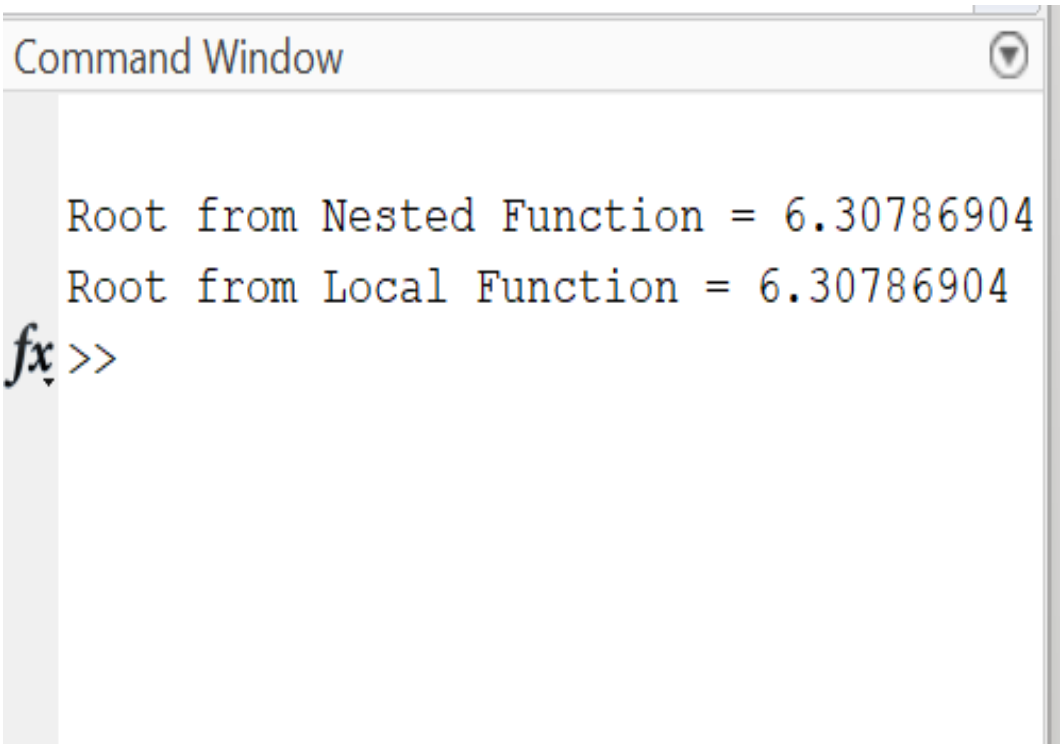

x0 = x1;


end


root = x1;


end
```

DISPLAY



RUNGE KUTTA METHOD

Given the Initial Value Problem (IVP):

$$\frac{d_y}{d_x} = x + y, y(0) = 1$$

Using the **4**th Order Runge-Kutta Method, approximate the value of $y\ at\ x\ =\ 0.2$ taking a step size $of\ h\ =\ 0.1$

Mathlab code

COMBINED MAIN, LOCAL AND NESTED FUNCTION

```
function main_rk4

clc;

clear;


x0 = 0;

y0 = 1;

h = 0.1;

xn = 0.2;


y1 = nestedRK4(x0,y0,h,xn);

fprintf('\nValue from Nested Function = %.8f\n',y1);


y2 = localRK4(x0,y0,h,xn);

fprintf('Value from Local Function = %.8f\n',y2);



function y = nestedRK4(x,y,h,xn)


while x < xn
```

```
k1 = h*(x + y);
k2 = h*((x+h/2) + (y+k1/2));
k3 = h*((x+h/2) + (y+k2/2));
k4 = h*((x+h) + (y+k3));

y = y + (k1 + 2*k2 + 2*k3 + k4)/6;
x = x + h;

end

end

end

function y = localRK4(x,y,h,xn)

while x < xn

k1 = h*(x + y);
k2 = h*((x+h/2) + (y+k1/2));
k3 = h*((x+h/2) + (y+k2/2));
k4 = h*((x+h) + (y+k3));

y = y + (k1 + 2*k2 + 2*k3 + k4)/6;
x = x + h;
```

```
end


end
```

DISPLAY

